ACIT 3855 – Service Oriented Architectures – Winter 2026

Data Storage in Microservices

Most services within a microservices based system need to store data. This data storage could be in a cache (if the data is transient), a data file (i.e., text, json or other format) or most likely a database (MySQL, PostgreSQL, SQL Server, Mongo, etc.). Please review Fowler's discussion on decentralized data management in microservices:

- https://martinfowler.com/articles/microservices.html#DecentralizedDataManagement

A monolithic application will have a single database for the entire application. In a microservices architecture, each service has its own independent data store following the Database Per Service Pattern. Please review this pattern:

- https://microservices.io/patterns/data/database-per-service.html

Note: If you remember our definition of a design pattern from ACIT 2515, roughly: "A design pattern is a general, reusable solution to a commonly occurring problem in software design."

Likewise, we will discuss various architectural patterns for a microservices architecture which can be defined as a "general, reusable solution to a commonly occurring problem in software architecture within a given context."

Data Storage in the Lesson 3 Lab

We will be initially using a file based database for Lab 3 (i.e., SQLite) and migrating to a database server in Lab 4 (i.e., MySQL).

Please consider these questions:

- What is the key difference between sqlite and MySQL?
- What are the advantages and disadvantages of using a database like sqlite?
- What are the advantages and disadvantages of using a database like MySQL?

We will also be using an Object Relational Mapper (ORM), specifically SQLAlchemy to interact with our database. An ORM allows us to maps Python objects to database tables and interact with those objects as we would any other object in Python. It also allows us to be database agnostic since the ORM provides services to create, update, delete and query objects from the database. This will make it easy to transition from a SQLite to a MySQL database.

You may or may not have covered this in ACIT 2515. Here is a brief tutorial on SQLAlchemy.

Let's say we have a table for a Weather Station's Temperature reading:

temperature_reading						
id	station_id	station_name	temp_c	batch_timestamp	reading_timestamp	date_created

This table could be created in an SQLite database (called readings.sqlite) by the following Python code:

```
import sqlite3

conn = sqlite3.connect('readings.sqlite')

c = conn.cursor()
c.execute('''
          CREATE TABLE temperature_reading
          (id INTEGER PRIMARY KEY ASC,
           station_id VARCHAR(250) NOT NULL,
           station_name VARCHAR(250) NOT NULL,
           temp_c NUMERIC NOT NULL,
           batch_timestamp VARCHAR(100) NOT NULL,
           reading_timestamp VARCHAR(100) NOT NULL,
           date_created VARCHAR(100) NOT NULL)
          ''')

conn.commit()
conn.close()
```

SQLAlchemy Declaratives

A SQLAlchemy declarative is mapping of a Python object to a row in a database table.

It looks like this:

```
from sqlalchemy import Column, Integer, String, DateTime, func
from base import Base
import datetime


class TemperatureReading(Base):
    """ Temperature Reading """

    __tablename__ = "temperature_reading"

    id                 = Column(Integer, primary_key=True)
    station_id         = Column(String(250), nullable=False)
    station_name       = Column(String(250), nullable=False)
    temp_c             = Column(Numeric(4, 1), nullable=False)
    batch_timestamp    = Column(String(100), nullable=False)
    reading_timestamp  = Column(String(100), nullable=False)
    date_created       = Column(DateTime,
                                nullable=False,
                                default=func.now())

    def to_dict(self):
        """ Dictionary Representation of a temperature reading """
        dict = {}
        dict['id'] = self.id
        dict['station_id'] = self.station_id
```

TemperatureReading extends a SQLAlchemy Base

Table Mapping

Mapping to the database table

Column Mapping

Mapping to the columns in the database table

Custom Method

Helper method to put the object's attributes into a Python dictionary. This Python dictionary can later be serialized into JSON

```
        dict['station_name'] = self.station_name
        dict['temperature_celsius'] = self.temp_c
        dict['batch_timestamp'] = self.batch_timestamp
        dict['reading_timestamp'] = self.reading_timestamp
        dict['date_created'] = self.date_created

        return dict
```

SQLAlchemy Sessions

We need a database session to add or query TemperatureReading objects to/from the database. Here are some examples:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from temperature_reading import TemperatureReading

…

# Example Add a Temperature Reading
session = DB_SESSION()

tr = TemperatureReading(station_id,
                        station_name,
                        temp_c,
                        batch_timestamp,
                        reading_timestamp)


session.add(tr)


session.commit()
session.close()

…

# Example Query for All Temperature Readings
session = DB_SESSION()

all_readings = session.query(TemperatureReading).all()
one_reading = session.query(TemperatureReading).filter(
                 TemperatureReading.id = query_id).first()


session.close()
…
```

Create a database session

Create a new TemperatureReading object (calls the Constructor).

Adds the TemperatureReading object to the database session

Commits the TemperatureReading object to the database and closes the

Create a database session

Queries for TemperatureReading objects from the database.

Close the session

```
# Can now access public attributes and methods on the
# TemperatureReading objects
print("Temp Reading is %f from station %s" % (one_reading.temp_c,
                                              one_reading.station_name))

# Create a list of Python dictionaries for all readings
all_readings_list = []

for reading in all_readings:
    all_readings_list.append(reading.to_dict())

…
```

Mapped columns are public instance variables

Useful to convert to Python objects when returned as a response message from a connexion endpoint